



MCU-Friendly Mamba
Optimizing Mamba Model Inference on Microcontrollers

Bartosz Drabiński¹

Supervisor: Bo Yang¹
Responsible Professor: Qing Wang¹

¹EEMCS, Delft University of Technology, The Netherlands

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
May 22, 2026

Name of the student: Bartosz Drabiński
Final project course: CSE3000 Research Project
Thesis committee: dr. Qing Wang, Mark Neerincx

An electronic version of this thesis is available at <http://repository.tudelft.nl/>.

Abstract

Machine learning is becoming increasingly pervasive. While state-of-the-art models continue to grow in size and computational cost, Edge AI targets devices with strict limits on processing power and memory. TinyML — the branch of machine learning focused on running models on microcontrollers (MCUs) — exemplifies these constraints.

Meeting these constraints requires new model designs and careful adaptation of existing architectures. The Mamba architecture, built around State-Space Models (SSMs), is a promising candidate due to its compact parameterization and strong performance on long-context tasks. In this paper we survey recent efforts to deploy Mamba on microcontrollers and propose architecture and optimization strategies tailored to MCU constraints.

Keywords: TinyML, Mamba, microcontroller, Edge AI

1 Introduction

Machine learning has recently enabled solutions to problems that were difficult or infeasible with traditional algorithms. This progress produced both large, general-purpose models (e.g., large language models) and many compact, task-specific models.

Most deployments still rely on centralized data centres, which are efficient but impose limitations: they require network connectivity, add latency, and raise privacy concerns.

Edge AI reduces these limitations by running inference locally on user devices. TinyML focuses on the extreme end of this spectrum, designing models that run on microcontrollers (MCUs). MCUs are severely resource-constrained: typical devices provide one or two CPU cores (clocked below 240 MHz) and on the order of 512 KB of RAM, so they cannot host large models or their full-precision weights. Low latency is also essential because many TinyML applications (for example, audio or video streams) require real-time processing.

Numerous model families have been developed for different trade-offs, including multilayer perceptrons (MLPs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers. The recently proposed Mamba architecture [?] has emerged as a promising alternative to Transformers for long-context tasks. Mamba combines ideas from CNNs and RNNs and leverages Selective State-Space Model (S4) blocks to achieve a smaller memory footprint while maintaining strong performance on long sequences.

Prior work has explored optimizations for Mamba such as quantization [], pruning, and knowledge distillation []. MambaLite-Micro [] implemented PyTorch-style inference for Mamba directly in C and ran it on ESP32-S3 and STM32 microcontrollers; this preserves compatibility but makes architecture changes and further optimizations cumbersome.

This paper asks the following question: How can Mamba be adapted and optimized for efficient inference on microcontrollers? To answer it, we evaluate deployment strategies, identify memory and latency bottlenecks, and measure

the impact of GPU-oriented optimizations when applied to MCUs.

Section ?? surveys related work and background. Section ?? describes the experimental setup. Section ?? presents our deployment pipeline. Section ?? reports the experimental results. Section ?? discusses ethical considerations. Section ?? interprets the findings. Finally, Section ?? concludes and outlines future work.

2 Related Work and Background

2.1 TinyML

TinyML concerns the design, optimization, and deployment of machine learning models on highly resource-constrained embedded devices such as microcontrollers. Common techniques include compact architecture design, pruning, weight-sharing, and quantization (both post-training quantization and quantization-aware training). Toolchain and compiler optimizations (for example, model flattening, operator fusion, and memory planning) are also essential to meet MCU resource limits. Prior work has shown successful TinyML applications in audio, gesture recognition, and sensor analytics, but extreme footprint reduction and reliable real-time inference remain active challenges.

2.2 Mamba architecture

The Mamba architecture is based on State-Space Models (SSMs), which capture long-range dependencies while avoiding the quadratic complexity of attention. A discrete-time SSM updates a hidden state and emits outputs as:

$$\begin{aligned}h_t &= \overline{A}h_{t-1} + \overline{B}x_t \\y_t &= C\overline{h}_t + D\overline{x}_t\end{aligned}$$

On parallel hardware, this operation can be implemented efficiently using scan (parallel-prefix) algorithms. However, SSM-based architectures pose deployment challenges: SSM layers are sensitive to quantization [?; ?], and there is limited standardized tooling and export paths (for example, ONNX) for Mamba-like models [?]. These issues complicate portability and optimization for constrained devices.

Despite these challenges, Mamba’s compact parameterization and strong long-context performance make it a promising candidate for TinyML.

2.3 Research gap

Two primary gaps motivate this work. First, robust quantization methods for SSM-based layers are still immature, making it difficult to deploy Mamba models in low-precision formats without significant accuracy loss. Second, the toolchains that produce MCU-ready binaries with minimal memory footprint are incomplete for SSM-style models. This paper targets both gaps by exploring architecture-level adjustments and MCU-focused optimization techniques.

3 Methodology

This section should describe the experimental framework and explain how the research question is investigated.

3.1 Dataset and Preprocessing

Describe:

- Dataset selection

3.2 Experimental Design

Describe:

- Quantization strategies
- Memory optimization strategies
- Ways of measuring used memory
- Testing strategies and reliability
- Repeated runs and statistical stability

3.3 Model Architecture

Describe:

- Changes made to the original
- Hyperparameter choices

3.4 Deployment Optimization

Describe:

- Deployment choices
- INT8, INT4 quantization
- Quantization-aware training
- Latency benchmarking

4 Embedded Deployment Setup

This section should describe the embedded-oriented deployment environment and evaluation setup.

4.1 Software Stack

Describe:

- No platform IO
- No ONNX
- C library
- Pytorch
- Brevitas
- Torchao
- Other relevant libraries

4.2 Deployment Measurements

Explain:

- Model size measurement
- Memory throughout runtime measurement
- Latency benchmarking methodology
- Inference execution setup

4.3 Reproducibility

Describe:

- Random seeds
- Repeated runs
- Configuration consistency
- Code availability

5 Results

This section presents the experimental results of the impact of different techniques on the accuracy and memory usage. Results should be supported by descriptive and clear figures.

5.1 Effect of Quantization on Memory and Accuracy

This subsection presents:

- Memory reductions across quantization strategies
- Accuracy changes induced by quantization
- Variance of performance across repeated runs

5.2 Embedded Efficiency Trade-offs

This subsection presents:

- Inference latency measurements
- Model size comparisons
- Accuracy-efficiency relationships

5.3 Effects of other memory optimizations on accuracy and latency

This subsection presents:

- Impacts of Low rank factorization

6 Responsible Research

This section discusses the ethical implications of the work, reproducibility considerations, and the role of AI-assisted tools in the development process.

6.1 Ethical Considerations

Key ethical aspects include:

- Possibility of reducing power usage
- Enabling bad actors to run dangerous ML models on embedded devices (listening)

6.2 Reproducibility

The following aspects support reproducibility:

- Publicly available dataset and deterministic preprocessing pipeline
- Ease of reproducing the model training environment
- Access to the microcontroller version used
- Consistent evaluation metrics across all experimental runs
- Code structured to enable rerunning of full experimental pipelines

6.3 Use of AI Tools

Artificial intelligence tools were used as supportive instruments during the development of this work. Their use was limited to improving code readability, assisting with debugging, and refining written text for clarity and grammatical correctness.

Specifically, AI assistance was used for:

- Debugging implementation issues in experimental code
- Improving clarity and structure of written explanations
- Assisting with formatting and consistency in LaTeX writing

All scientific decisions, experimental design choices, and interpretation of results were made independently by the author. AI tools were not used to generate or alter experimental outcomes, results, or conclusions.

7 Discussion

This section interprets the experimental findings and discusses their implications for embedded trustworthy AI systems. It also discusses the limitations of the approach.

7.1 Trade offs between accuracy, memory efficiency, and latency

Discuss:

- How making models smaller reduces accuracy of them
- How some optimizations can be made that hurt latency

7.2 How Mamba model performs

Discuss:

- The performance of Mamba compared to SOTA for the dataset

7.3 Limitations

Discuss:

- Dataset limitations
- More quantization possible
- Hardware specific implementation
- Difficulty of comparison with other models
- Lack of standardized tooling

8 Conclusions and Future Work

This section should directly answer the research question and summarize the main findings.

The conclusion should:

- Restate the research objective
- Summarize the most important findings
- Explain the memory limitations of Mamba models on MCUs
- Highlight key trade-offs identified in the experiments

Future work may discuss:

- Other memory optimization strategies
- More standardized deployment (LiteRT custom operation)
- Energy-aware optimization

The conclusion should remain concise and readable as a standalone section.